

Document de cadrage – Moulti Game

Julia Grossi

Sommaire

1. **Présentation générale**
 - 1.1 Contexte du projet
 - 1.2 Informations générales
2. **Contexte et problématique**
3. **Objectifs du projet**
 - 3.1 Objectif principal
 - 3.2 Objectifs techniques
4. **Présentation fonctionnelle**
 - 4.1 Public cible
 - 4.2 Proposition de valeur
 - 4.3 Mini-jeux disponibles
5. **Périmètre du projet**
 - 5.1 Périmètre fonctionnel
 - 5.2 Hors périmètre
6. **Parcours utilisateur**
7. **Organisation fonctionnelle de l'application**
8. **Priorisation des fonctionnalités**
 - 8.1 Fonctionnalités indispensables (Must Have)
 - 8.2 Fonctionnalités souhaitables (Should Have)
 - 8.3 Fonctionnalités d'évolution (Nice to Have)
9. **Architecture technique**
 - 9.1 Vue d'ensemble
 - 9.2 Fonctionnement général
 - 9.3 Architecture logique
10. **Choix technologiques**
 - 10.1 Frontend
 - 10.2 Backend

10.3 Base de données

10.4 Conteneurisation et versionnement

11. Modélisation des données

11.1 Modèle conceptuel de données (MCD)

11.2 Modèle logique de données (MLD)

11.3 Modèle physique de données (MPD)

12. API REST

12.1 Authentification

12.2 Gestion des scores

12.3 Analyse des performances

12.4 Route de supervision

13. Sécurité

13.1 Authentification des utilisateurs

13.2 Protection des mots de passe

13.3 Protection des données

13.4 Validation des données

13.5 Protection de la clé OpenAI

14. Fonctionnalité d'intelligence artificielle

14.1 Objectif

14.2 Données analysées

14.3 Fonctionnement

14.4 Structure de la réponse

14.5 Valeur ajoutée

15. Contraintes et risques

15.1 Contraintes du projet

15.2 Risques identifiés

15.3 Réponses apportées

16. Critères de réussite

17. Pistes d'évolution

18. Conclusion



1. Présentation générale

1.1 Contexte du projet

Dans le cadre de la formation **Ynov Connect – Master 2 Développement Full Stack**, il est demandé de concevoir une application web complète mettant en œuvre une architecture client-serveur ainsi qu'une base de données relationnelle. Ce projet individuel doit démontrer la capacité à développer une application full stack en intégrant les différentes couches techniques d'une solution moderne : interface utilisateur, API, base de données et services complémentaires.

Le projet retenu, **Moulti Game**, consiste à développer une plateforme web regroupant plusieurs mini-jeux accessibles depuis une interface unique. Au-delà de l'aspect ludique, cette application permet de gérer des comptes utilisateurs, d'enregistrer les résultats des parties et d'offrir une analyse personnalisée des performances grâce à l'intégration d'une intelligence artificielle.

Le projet est développé pour être exécuté localement et son code source est versionné sur GitHub conformément aux exigences.

1.2 Informations générales

Élément	Description
Nom du projet	Moulti Game

Formation	Ynov Connect – Master 2 Développement Full Stack
Type de projet	Projet individuel
Architecture	Application Web Full Stack
Langages principaux	HTML, CSS, JavaScript, Node.js
Base de données	PostgreSQL
API	REST
Authentification	JWT
Intelligence artificielle	API OpenAI
Déploiement attendu	Exécution locale

2. Contexte et problématique

Aujourd'hui, de nombreux mini-jeux sont disponibles gratuitement sur Internet. Toutefois, ces jeux fonctionnent généralement de manière indépendante et ne proposent ni espace personnel, ni historique des parties, ni suivi des performances du joueur. Une fois la partie terminée, les résultats sont souvent perdus et aucune analyse n'est proposée afin d'accompagner la progression de l'utilisateur.

L'objectif de **Moulti Game** est de répondre à cette problématique en proposant une plateforme unique regroupant plusieurs mini-jeux au sein d'une même application. Chaque joueur possède un compte personnel lui permettant de retrouver ses différentes parties, de consulter l'évolution de ses scores et d'obtenir des conseils personnalisés générés automatiquement grâce à une intelligence artificielle.

Ce projet permet également de mettre en œuvre l'ensemble des compétences attendues dans le cadre d'un développement full stack. Il mobilise aussi bien la conception d'une interface utilisateur que le développement d'une API REST sécurisée, la gestion d'une base de données relationnelle ainsi que l'intégration d'un service externe.

3. Objectifs du projet

3.1 Objectif principal

L'objectif principal de Moulti Game est de concevoir une application web full stack permettant à un utilisateur de créer un compte, d'accéder à plusieurs mini-jeux, d'enregistrer automatiquement ses scores et de consulter l'historique de ses performances.

L'application doit également intégrer une fonctionnalité d'intelligence artificielle capable d'analyser les résultats enregistrés afin de produire un retour personnalisé mettant en évidence les points forts, les axes de progression et les prochains objectifs du joueur.

3.2 Objectifs techniques

Afin de répondre aux besoins identifiés, plusieurs objectifs techniques ont été définis :

- concevoir une architecture claire séparant le frontend, le backend et la base de données ;
- développer une API REST permettant la communication entre les différentes couches de l'application ;
- mettre en place une authentification sécurisée reposant sur des JSON Web Tokens (JWT) ;
- stocker les données utilisateurs et les scores dans une base PostgreSQL ;
- intégrer un service d'intelligence artificielle via l'API OpenAI ;
- assurer un fonctionnement complet de bout en bout, depuis l'interface utilisateur jusqu'à la persistance des données.

4. Présentation fonctionnelle

4.1 Public cible

Moulti Game s'adresse principalement aux utilisateurs souhaitant accéder rapidement à plusieurs mini-jeux depuis un navigateur web sans avoir à installer d'application spécifique.

La plateforme cible notamment :

- les joueurs occasionnels recherchant des parties rapides ;
- les étudiants souhaitant disposer d'une plateforme ludique accessible depuis n'importe quel ordinateur ;
- les utilisateurs intéressés par le suivi de leurs performances et de leur progression au fil du temps.

4.2 Proposition de valeur

Contrairement aux nombreux mini-jeux disponibles individuellement sur Internet, Moulti Game centralise plusieurs expériences de jeu au sein d'une même plateforme.

Chaque utilisateur bénéficie d'un espace personnel lui permettant de retrouver son historique, de suivre son évolution et d'obtenir une analyse intelligente de ses résultats. Cette approche transforme une simple succession de parties en une véritable expérience de progression.

La principale valeur ajoutée de l'application repose donc sur :

- la centralisation de plusieurs mini-jeux dans une interface unique ;
- la conservation des scores de chaque joueur ;
- le suivi de la progression dans le temps ;

- l'analyse automatique des performances grâce à l'intelligence artificielle.

4.3 Mini-jeux disponibles

La première version de l'application intègre cinq mini-jeux développés en JavaScript :

- Snake ;
- Simon ;
- Le Pendu ;
- Devine le Nombre ;
- Puissance 4.

Chaque jeu fonctionne indépendamment mais partage le même système d'authentification ainsi que le même mécanisme d'enregistrement des scores.

5. Périmètre du projet

5.1 Périmètre fonctionnel

Le périmètre du projet regroupe l'ensemble des fonctionnalités nécessaires pour offrir une expérience utilisateur complète, depuis la création d'un compte jusqu'à l'analyse des performances réalisées au cours des différentes parties.

L'application permettra à un utilisateur de créer un compte personnel, de s'authentifier de manière sécurisée puis d'accéder aux différents mini-jeux disponibles. À la fin de chaque partie, le score obtenu sera automatiquement enregistré dans la base de données afin de constituer un historique personnel.

L'utilisateur pourra consulter cet historique, filtrer les résultats selon le jeu concerné et supprimer tout ou partie des scores enregistrés. Enfin, une fonctionnalité basée sur l'intelligence artificielle analysera les performances afin de fournir une synthèse personnalisée mettant en évidence les points forts et les axes de progression.

Les principales fonctionnalités retenues sont les suivantes :

- création d'un compte utilisateur ;
- connexion sécurisée ;
- authentification par JWT ;
- accès protégé aux fonctionnalités nécessitant une identification ;

- lancement des différents mini-jeux ;
- enregistrement automatique des scores ;
- consultation de l'historique des parties ;
- filtrage des résultats par jeu ;
- suppression de l'historique complet ou partiel ;
- génération d'une analyse des performances par intelligence artificielle.

5.2 Hors périmètre

Afin de respecter le temps imparti et de concentrer les efforts sur les fonctionnalités essentielles, plusieurs éléments ont volontairement été exclus de cette première version.

Le projet ne prévoit notamment pas :

- un déploiement sur une infrastructure de production ;
- un mode multijoueur en temps réel ;
- un classement mondial entre joueurs ;
- l'export des données au format CSV ;
- une application mobile native ;
- un système complet d'intégration et de déploiement continu (CI/CD).

Ces fonctionnalités pourront néanmoins constituer des pistes d'évolution dans les versions futures.

6. Parcours utilisateur

Le fonctionnement de l'application a été pensé afin d'offrir une expérience simple et intuitive.

Lorsqu'un utilisateur accède à l'application, il est invité à créer un compte ou à se connecter à l'aide de ses identifiants. Une fois authentifié, il accède à un espace personnel regroupant les différents mini-jeux disponibles.

Après avoir sélectionné un jeu, la partie se déroule directement dans le navigateur. À son terme, le score obtenu est transmis au serveur via l'API REST, puis enregistré dans la base de données PostgreSQL.

L'utilisateur peut ensuite consulter son historique de parties afin de visualiser ses performances passées. Depuis cette même interface, il lui est également possible de demander une analyse générée par intelligence artificielle, qui synthétise ses résultats et propose des recommandations adaptées à son niveau.

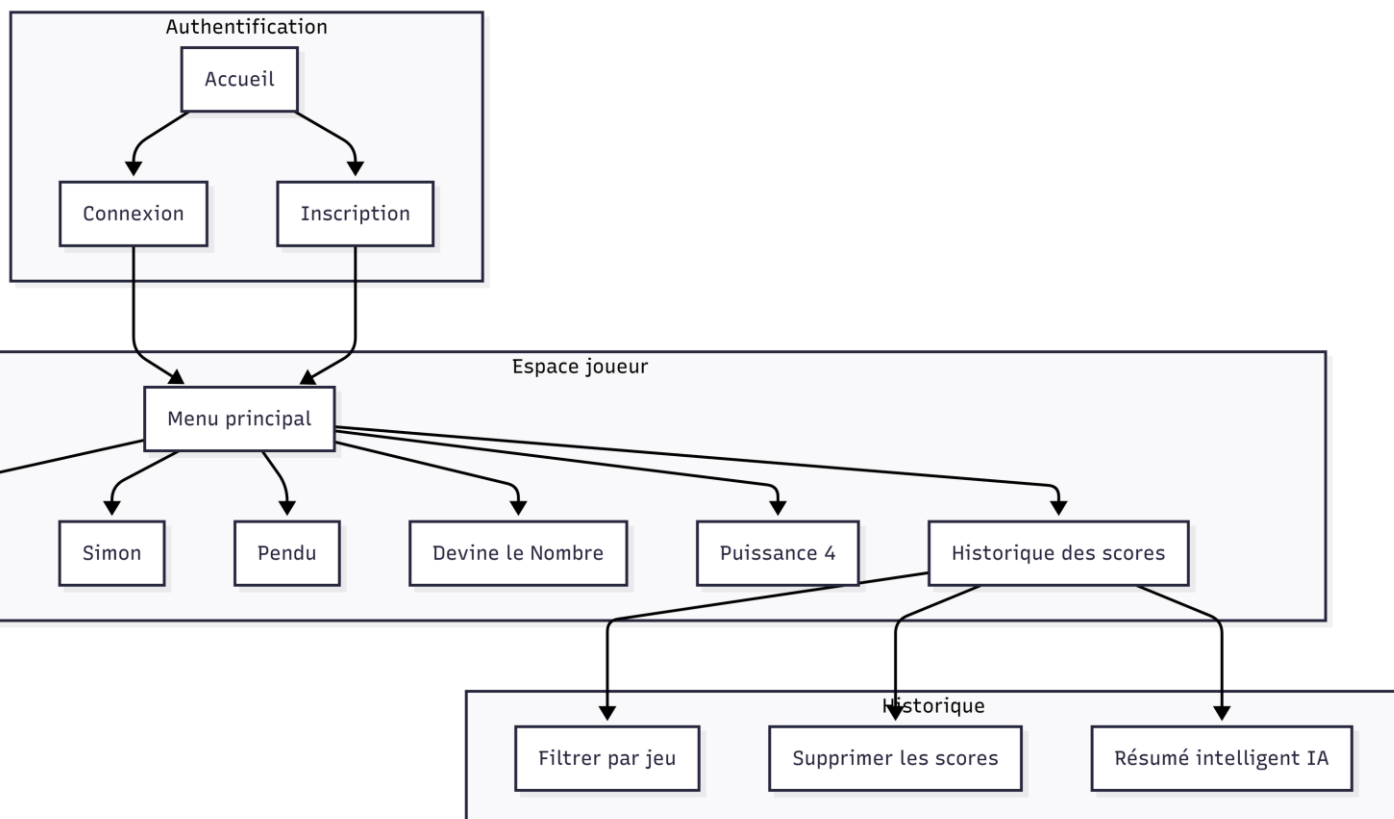
Le parcours utilisateur peut être résumé de la façon suivante :

1. arrivée sur la page d'accueil ;
2. création d'un compte ou connexion ;
3. authentification via JWT ;
4. accès à l'espace personnel ;
5. sélection d'un mini-jeu ;
6. déroulement de la partie ;
7. enregistrement automatique du score ;
8. consultation de l'historique ;
9. génération d'une analyse intelligente des performances.

7. Organisation fonctionnelle de l'application

L'application est organisée autour d'un espace utilisateur unique donnant accès aux différentes fonctionnalités après authentification.

L'arborescence générale est la suivante :



Cette organisation permet de séparer clairement les fonctionnalités d'authentification, de jeu et de consultation des données.

8. Priorisation des fonctionnalités

Le développement de l'application a été réalisé en distinguant les fonctionnalités indispensables des améliorations pouvant être intégrées ultérieurement.

Fonctionnalités indispensables (Must Have)

Les fonctionnalités suivantes sont essentielles au bon fonctionnement de l'application :

- inscription et connexion des utilisateurs ;
- authentification sécurisée par JWT ;
- accès aux différents mini-jeux ;
- enregistrement automatique des scores ;
- consultation de l'historique des parties ;
- génération d'un résumé des performances grâce à l'intelligence artificielle.

Ces fonctionnalités constituent le cœur du projet et répondent directement aux

Fonctionnalités souhaitables (Should Have)

Certaines fonctionnalités améliorent le confort d'utilisation sans être indispensables :

- filtrage de l'historique par type de jeu ;
- suppression sélective ou complète des scores enregistrés.

Fonctionnalités d'évolution (Nice to Have)

Enfin, plusieurs fonctionnalités pourront être envisagées dans une version ultérieure du projet :

- classement global entre tous les joueurs ;
- export des résultats ;
- statistiques graphiques ;
- ajout de nouveaux mini-jeux ;
- mode multijoueur ;
- déploiement en ligne.

objectifs fixés lors de sa conception.

9. Architecture technique

9.1 Vue d'ensemble

L'application Moulti Game repose sur une architecture **client-serveur** classique, permettant de séparer les responsabilités entre l'interface utilisateur, la logique métier et la persistance des données.

Cette séparation présente plusieurs avantages. Elle facilite la maintenance de l'application, améliore sa modularité et permet à chaque composant d'évoluer indépendamment des autres. Le frontend est uniquement responsable de l'affichage et des interactions avec l'utilisateur, tandis que le backend centralise les traitements métiers, la sécurité et les échanges avec la base de données.

L'ensemble des communications entre le navigateur et le serveur est réalisé au travers d'une API REST utilisant le format JSON.

L'architecture générale est composée des éléments suivants :

- un frontend développé en HTML, CSS et JavaScript ;
- un backend développé avec Node.js et Express ;
- une base de données PostgreSQL ;
- un service d'intelligence artificielle accessible via l'API OpenAI.

9.2 Fonctionnement général

Lorsqu'un utilisateur effectue une action dans l'application, celle-ci est d'abord traitée par le frontend. Si cette action nécessite des données ou une modification de la base, une requête HTTP est envoyée vers l'API REST.

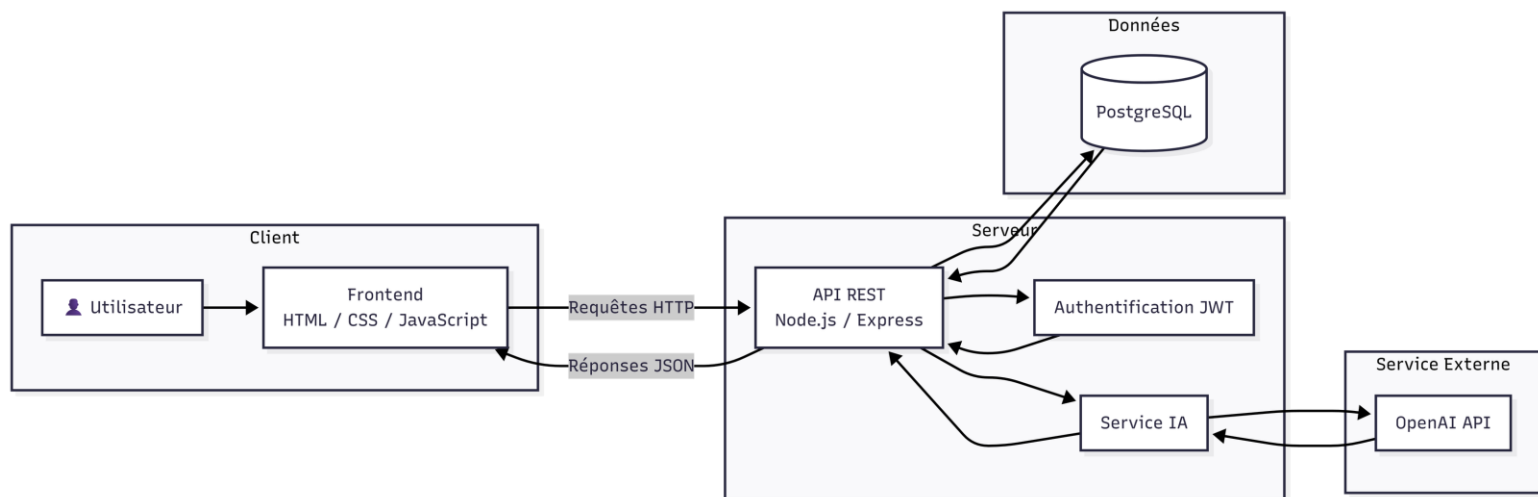
Le backend vérifie alors les droits de l'utilisateur, applique les traitements nécessaires puis communique avec PostgreSQL afin de récupérer ou d'enregistrer les informations demandées.

Dans le cas de la fonctionnalité d'analyse intelligente, le backend construit également une requête destinée à l'API OpenAI. La réponse obtenue est ensuite retransmise au frontend afin d'être affichée à l'utilisateur.

Cette organisation permet de ne jamais exposer directement la clé API OpenAI au navigateur et garantit une meilleure sécurité de l'application.

9.3 Architecture logique

Le schéma suivant illustre les échanges entre les différents composants.



10. Choix technologiques

Le choix des technologies utilisées s'est porté sur des solutions largement répandues dans le développement web moderne. L'objectif était de privilégier des outils simples à mettre en œuvre tout en restant représentatifs d'une architecture professionnelle.

10.1 Frontend

L'interface utilisateur est développée exclusivement en **HTML**, **CSS** et **JavaScript Vanilla**.

Ce choix permet de conserver une architecture légère, sans dépendance à un framework JavaScript tel que React ou Vue.js. Les mini-jeux étant relativement simples, l'utilisation d'un framework complet n'aurait pas apporté de bénéfice significatif tout en augmentant la complexité du projet.

Les échanges avec le serveur sont réalisés grâce à l'API **Fetch**, qui permet d'envoyer des requêtes HTTP de manière simple et standardisée.

Les principales responsabilités du frontend sont :

- afficher les différentes pages de l'application ;
- gérer les interactions avec l'utilisateur ;
- lancer les mini-jeux ;
- communiquer avec l'API REST ;
- présenter les données reçues du backend.

10.2 Backend

Le serveur applicatif est développé avec **Node.js** en utilisant le framework **Express**.

Express facilite la création d'une API REST en proposant un système simple de routes, de middlewares et de gestion des requêtes HTTP.

Le backend constitue le cœur de l'application. Il est responsable :

- de l'authentification des utilisateurs ;
- de la génération des JWT ;
- de la validation des requêtes ;
- de la communication avec PostgreSQL ;
- de l'appel à l'API OpenAI ;
- de la sécurisation des données.

Cette centralisation de la logique métier permet de maintenir un frontend relativement simple et limite les risques liés à la sécurité.

10.3 Base de données

Les données sont stockées dans une base **PostgreSQL**.

Ce système de gestion de bases de données a été retenu pour plusieurs raisons :

- excellente fiabilité ;
- très bonnes performances ;
- gestion avancée des contraintes relationnelles ;
- compatibilité avec Node.js ;
- prise en charge du type **JSONB**.

Le type **JSONB** est notamment utilisé afin de stocker des informations complémentaires propres à chaque mini-jeu sans avoir à modifier le schéma de la base à chaque nouvelle évolution.

Cette approche offre davantage de flexibilité pour les futures versions du projet.

10.4 Conteneurisation et versionnement

Le projet est exécuté localement grâce à **Docker Compose**, ce qui permet de démarrer rapidement l'ensemble des services nécessaires sans installation complexe.

Le code source est versionné avec **Git** puis hébergé sur **GitHub**, offrant un historique complet des modifications ainsi qu'une meilleure traçabilité du développement.

Cette organisation facilite également la collaboration et prépare le projet à un éventuel déploiement futur.

11. Modélisation des données

La base de données a été conçue afin de répondre aux besoins fonctionnels de l'application tout en restant volontairement simple.

Deux entités principales ont été identifiées : les joueurs et les scores.

Chaque joueur possède un compte unique et peut enregistrer un nombre illimité de parties.

Chaque score est obligatoirement associé à un seul joueur.

11.1 Modèle conceptuel de données (MCD)

Le modèle conceptuel fait apparaître deux entités principales :

PLAYER		
UUID	id	PK
VARCHAR	username	
VARCHAR	email	
VARCHAR	role	
TEXT	password_hash	
TIMESTAMPTZ	created_at	



PLAYER_SCORE		
BIGSERIAL	id	PK
UUID	player_id	FK
VARCHAR	game_name	
INTEGER	score_value	
JSONB	metadata	
TIMESTAMPTZ	played_at	

Player

Cette entité représente un utilisateur authentifié de l'application.

Elle contient notamment :

- l'identifiant du joueur ;
- son pseudonyme ;
- son adresse électronique ;
- son mot de passe chiffré ;
- sa date de création.

PlayerScore

Cette entité représente une partie réalisée par un joueur.

Elle contient :

- l'identifiant de la partie ;
- le jeu concerné ;
- le score obtenu ;
- les éventuelles informations complémentaires ;
- la date de réalisation.

La relation entre ces deux entités est de type **1,N** :

Un joueur peut posséder plusieurs scores, tandis qu'un score appartient obligatoirement à un seul joueur.

11.2 Modèle logique de données (MLD)

Le modèle logique se traduit par deux tables relationnelles :

PLAYERS		
UUID	id	PK
VARCHAR	username	
VARCHAR	email	UK
VARCHAR	role	
TEXT	password_hash	
TIMESTAMPTZ	created_at	



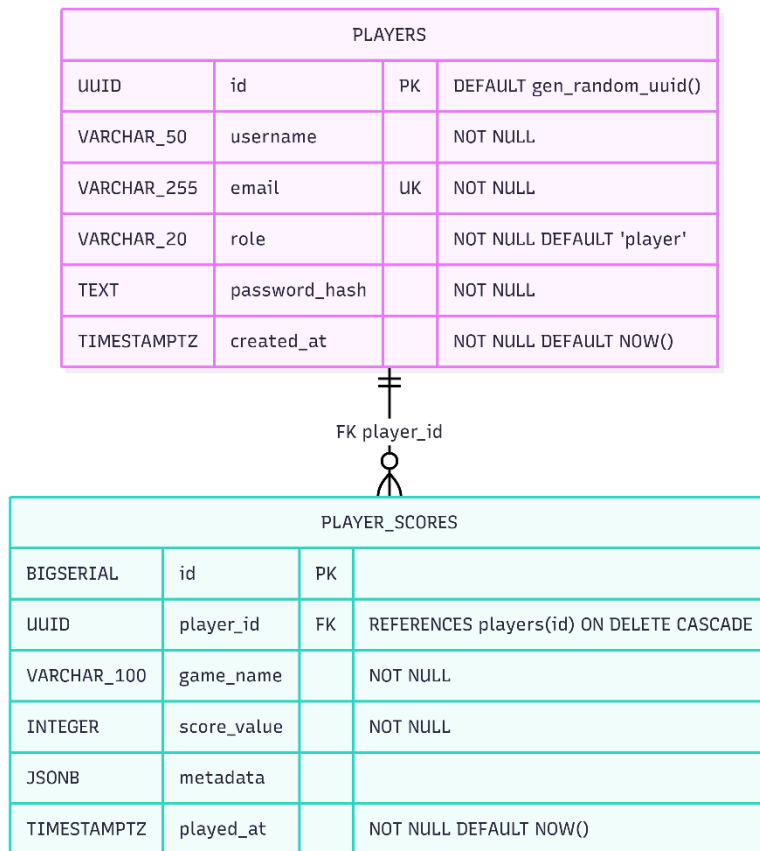
PLAYER_SCORES		
BIGSERIAL	id	PK
UUID	player_id	FK
VARCHAR	game_name	
INTEGER	score_value	
JSONB	metadata	
TIMESTAMPTZ	played_at	

- **players**
- **player_scores**

La table **player_scores** possède une clé étrangère pointant vers la table **players**, garantissant ainsi l'intégrité référentielle.

Cette organisation facilite les recherches de scores par utilisateur tout en évitant la duplication des informations relatives aux joueurs.

11.3 Modèle physique de données (MPD)



Le choix d'un identifiant UUID pour les joueurs permet également de limiter les risques liés à l'exposition d'identifiants séquentiels.

Le modèle physique est implémenté dans PostgreSQL.

La table **players** stocke les informations d'identification des utilisateurs tandis que la table **player_scores** enregistre les résultats des différentes parties.

Les suppressions sont propagées automatiquement grâce à une contrainte **ON DELETE CASCADE**, garantissant qu'aucun score orphelin ne subsiste lorsqu'un joueur est supprimé.

12. API REST

L'ensemble des échanges entre le frontend et le backend est réalisé au moyen d'une API REST. Cette architecture permet de découpler complètement l'interface utilisateur de la logique métier tout en facilitant la maintenance et l'évolution de l'application.

Toutes les données sont échangées au format **JSON**, un standard largement utilisé dans le développement web moderne. Chaque ressource de l'application est accessible via un ensemble d'URL clairement identifiées, associées aux méthodes HTTP appropriées.

L'API est organisée en plusieurs groupes de fonctionnalités : l'authentification, la gestion des scores, l'analyse des performances par intelligence artificielle ainsi qu'une route de contrôle permettant de vérifier le bon fonctionnement

du

serveur.

12.1 Authentification

Les routes d'authentification permettent aux utilisateurs de créer un compte, de se connecter et de récupérer les informations de leur session.

Méthode	Endpoint	Description
POST	/api/auth/register	Création d'un nouveau compte utilisateur
POST	/api/auth/login	Authentification et génération d'un JWT
GET	/api/auth/me	Récupération des informations de l'utilisateur connecté

Lors de la connexion, le serveur vérifie les identifiants fournis par l'utilisateur. Si ceux-ci sont valides, un **JSON Web Token (JWT)** est généré puis renvoyé au frontend.

Ce jeton est ensuite transmis lors des requêtes nécessitant une authentification afin d'identifier l'utilisateur sans conserver de session côté serveur.

12.2 Gestion des scores

Les routes relatives aux scores permettent d'enregistrer les résultats obtenus lors des différentes parties et de consulter l'historique personnel du joueur.

Méthode	Endpoint	Description
POST	/api/scores	Enregistrer un nouveau score
GET	/api/scores/games	Récupérer la liste des jeux disponibles

GET	/api/scores/history	Consulter l'historique des scores
DELETE	/api/scores/history	Supprimer tout ou partie de l'historique

Chaque score est automatiquement associé à l'utilisateur authentifié grâce aux informations contenues dans le JWT. Cette approche garantit qu'un joueur ne peut consulter ou modifier que ses propres données.

12.3 Analyse des performances

Une route spécifique est dédiée à la génération du résumé intelligent des performances.

Méthode	Endpoint	Description
GET	/api/insights/performance-summary	Génération d'une synthèse personnalisée via l'IA

Cette route est protégée par authentification. Avant d'interroger l'API OpenAI, le backend récupère les statistiques du joueur puis construit un prompt contenant les informations utiles à l'analyse.

12.4 Route de supervision

Une route supplémentaire permet de vérifier rapidement que le serveur est opérationnel.

Méthode	Endpoint	Description
GET	/health	Vérification de l'état du serveur

Cette route est principalement utilisée lors du développement afin de contrôler le bon démarrage de l'application.

13. Sécurité

La sécurité constitue un élément essentiel du projet. Même si l'application est destinée à un usage académique et à une exécution locale, plusieurs mécanismes ont été mis en place afin de respecter les bonnes pratiques de développement.

13.1 Authentification des utilisateurs

L'accès aux fonctionnalités nécessitant un compte est protégé par un système d'authentification reposant sur les **JSON Web Tokens (JWT)**.

Après une connexion réussie, le serveur génère un jeton signé contenant l'identifiant de l'utilisateur. Ce jeton est ensuite envoyé par le client dans l'en-tête Authorization de chaque requête protégée.

Le backend vérifie systématiquement la validité de ce jeton avant d'autoriser l'accès aux ressources demandées.

Cette méthode présente plusieurs avantages :

- absence de session stockée côté serveur ;
- architecture facilement scalable ;
- authentification simple et performante.

13.2 Protection des mots de passe

Les mots de passe des utilisateurs ne sont jamais enregistrés en clair dans la base de données.

Avant leur stockage, ils sont chiffrés à l'aide d'un algorithme de hachage sécurisé. Ainsi, même en cas d'accès non autorisé à la base de données, les mots de passe d'origine ne peuvent pas être retrouvés.

13.3 Protection des données

Toutes les opérations réalisées sur les scores sont automatiquement liées à l'utilisateur authentifié.

Le backend ignore les éventuels identifiants transmis par le client et utilise exclusivement les informations extraites du JWT afin de déterminer le propriétaire des données.

Cette approche empêche un utilisateur de consulter ou de modifier les scores appartenant à un autre compte.

13.4 Validation des données

Avant d'être enregistrées dans la base de données, les informations reçues par l'API sont vérifiées afin de garantir leur cohérence.

Cette validation permet notamment :

- de contrôler les types de données reçus ;
- de vérifier la présence des champs obligatoires ;
- de limiter les risques d'erreurs applicatives ;
- de prévenir certaines attaques reposant sur des données malformées.

13.5 Protection de la clé OpenAI

L'intégration de l'intelligence artificielle repose sur une clé d'API confidentielle.

Pour garantir sa sécurité, cette clé n'est jamais transmise au navigateur. Elle est stockée dans les variables d'environnement du serveur puis utilisée exclusivement par le backend lors des appels à l'API OpenAI.

Cette architecture empêche tout utilisateur d'accéder directement aux informations d'authentification du service.

14. Fonctionnalité d'intelligence artificielle

L'une des principales valeurs ajoutées de Moulti Game réside dans l'intégration d'une fonctionnalité d'intelligence artificielle permettant d'analyser automatiquement les performances du joueur.

Contrairement à un simple historique de scores, cette fonctionnalité transforme les données enregistrées en recommandations personnalisées afin d'aider l'utilisateur à suivre sa progression.

14.1 Objectif

L'objectif de cette fonctionnalité est de fournir une interprétation des résultats obtenus au cours des différentes parties.

À partir des statistiques enregistrées dans la base de données, l'intelligence artificielle est capable d'identifier les jeux sur lesquels le joueur progresse, ceux pour lesquels il rencontre davantage de difficultés ainsi que les axes d'amélioration envisageables.

L'utilisateur obtient ainsi un retour personnalisé plutôt qu'une simple liste de scores.

14.2 Données analysées

Avant chaque appel à l'API OpenAI, le backend agrège les informations disponibles afin de construire un résumé statistique.

Les données exploitées comprennent notamment :

- le nombre total de parties jouées ;
- le meilleur score obtenu ;
- le score moyen ;
- l'évolution récente des performances ;
- le dernier jeu pratiqué ;
- la date de la dernière session.

Ces informations permettent à l'intelligence artificielle de produire une analyse cohérente sans avoir à traiter directement l'ensemble de l'historique.

14.3 Fonctionnement

Le processus d'analyse se déroule selon les étapes suivantes :

1. l'utilisateur demande un résumé de ses performances depuis l'interface ;
2. le frontend envoie une requête au backend ;
3. le backend récupère les statistiques dans PostgreSQL ;
4. un prompt est généré à partir de ces données ;
5. le prompt est transmis à l'API OpenAI ;
6. la réponse est analysée puis convertie au format JSON ;
7. le frontend affiche le résumé généré.

Cette architecture garantit que l'ensemble des traitements sensibles reste exécuté côté serveur.

14.4 Structure de la réponse

La réponse renvoyée par le service d'intelligence artificielle est structurée afin d'être directement exploitable par le frontend.

Elle contient notamment les éléments suivants :

- une vue d'ensemble des performances ;
- les principaux points forts du joueur ;
- les axes de progression identifiés ;
- le mini-jeu recommandé ;
- une justification de cette recommandation ;

- un objectif concret à atteindre lors des prochaines parties.

Ce format facilite la présentation des résultats dans l'interface utilisateur.

14.5 Valeur ajoutée

L'intégration de l'intelligence artificielle constitue un véritable différenciateur pour l'application.

Au lieu de simplement afficher des données brutes, Moulti Game est capable d'interpréter les performances du joueur et de produire un retour personnalisé, compréhensible et orienté vers la progression.

Cette fonctionnalité illustre concrètement l'utilisation d'un service d'intelligence artificielle au sein d'une architecture full stack moderne, tout en apportant une réelle valeur fonctionnelle à l'utilisateur.

15. Contraintes et risques

15.1 Contraintes du projet

Le projet Moulti Game s'inscrit dans un cadre académique précis. Il doit être réalisé individuellement, dans un temps limité, tout en respectant les consignes attendues pour un projet full stack.

La principale contrainte est donc de proposer une application suffisamment complète pour démontrer la maîtrise des compétences techniques, sans chercher à développer une plateforme trop ambitieuse.

Le projet doit pouvoir être exécuté localement, sans nécessiter de déploiement en production. Le code source doit également être disponible sur un dépôt GitHub public afin de permettre son évaluation.

Enfin, la fonctionnalité d'intelligence artificielle doit être réellement implémentée et démontrable, et non simplement évoquée dans la documentation.

15.2 Risques identifiés

Plusieurs risques ont été identifiés au cours de la conception du projet.

Le premier risque concerne la dispersion du temps de développement. L'intégration de plusieurs mini-jeux peut rapidement devenir chronophage, notamment si chaque jeu nécessite une logique spécifique de score, d'affichage et de sauvegarde.

Un second risque concerne l'hétérogénéité des données. Chaque mini-jeu peut produire des informations différentes, ce qui peut complexifier l'enregistrement des scores et leur analyse.

La fonctionnalité IA constitue également une dépendance externe. Son bon fonctionnement repose sur la disponibilité de l'API OpenAI et sur la bonne configuration de la clé d'accès.

Enfin, un risque existe entre le périmètre initialement prévu et le périmètre réellement finalisé. Il est donc important de documenter clairement les fonctionnalités effectivement implémentées.

15.3 Réponses apportées

Pour limiter ces risques, le périmètre du projet a été volontairement maîtrisé. Les fonctionnalités essentielles ont été priorisées afin de garantir un parcours utilisateur complet avant d'ajouter des fonctionnalités secondaires.

Le format d'enregistrement des scores a également été unifié afin de permettre une gestion cohérente des données, même lorsque les mini-jeux possèdent des règles différentes.

L'appel à l'intelligence artificielle est centralisé côté backend, ce qui simplifie la maintenance et sécurise la clé API.

Enfin, le document de cadrage sert de référence afin d'assurer la cohérence entre les objectifs initiaux, les choix techniques et l'application réellement développée.

16. Critères de réussite

Le projet sera considéré comme réussi si l'application permet à un utilisateur de réaliser un parcours complet, depuis la création de son compte jusqu'à la consultation de ses performances.

Les critères principaux sont les suivants :

- un utilisateur peut créer un compte ;
- un utilisateur peut se connecter de manière sécurisée ;
- les routes protégées sont accessibles uniquement après authentification ;
- plusieurs mini-jeux sont disponibles depuis l'interface ;
- les scores sont correctement enregistrés en base de données ;

- l'historique des scores est consultable ;
- les scores peuvent être filtrés ou supprimés ;
- la fonctionnalité IA génère une analyse exploitable ;
- le projet peut être lancé localement ;
- le code source est disponible sur GitHub.

Ces critères permettent de vérifier que l'application répond à la fois aux attentes fonctionnelles et aux exigences techniques du projet.

17. Pistes d'évolution

Même si la première version de Moulti Game répond aux objectifs principaux, plusieurs améliorations peuvent être envisagées dans une version future.

Une première évolution consisterait à ajouter un classement global entre les joueurs. Cette fonctionnalité renforcerait l'aspect compétitif de la plateforme et encouragerait les utilisateurs à améliorer leurs scores.

Il serait également possible d'ajouter des tableaux de bord statistiques plus visuels, avec des graphiques permettant de suivre la progression dans le temps.

De nouveaux mini-jeux pourraient être intégrés afin d'enrichir l'expérience utilisateur.

À plus long terme, un mode multijoueur pourrait être envisagé pour certains jeux, notamment Puissance 4.

Enfin, un déploiement en ligne permettrait de rendre l'application accessible à un public plus large et de tester son fonctionnement dans un environnement proche de la production.

18. Conclusion

Moulti Game est un projet full stack conçu pour répondre à un objectif simple : proposer une plateforme web ludique regroupant plusieurs mini-jeux tout en intégrant une gestion des utilisateurs, une persistance des scores et une analyse intelligente des performances.

Le projet met en œuvre les principales briques d'une application web moderne : une interface frontend, une API REST, une base de données

relationnelle, une authentification sécurisée et l'intégration d'un service externe d'intelligence artificielle.

Son périmètre a été volontairement maîtrisé afin de garantir un fonctionnement complet de bout en bout. Cette approche permet de démontrer la cohérence entre les besoins fonctionnels, les choix techniques et l'implémentation finale.

Grâce à son historique de scores et à son résumé IA, Moulti Game dépasse le simple cadre d'une collection de mini-jeux. L'application propose une expérience personnalisée, orientée vers la progression du joueur, tout en constituant un support pertinent pour démontrer les compétences acquises en développement full stack.